UNIVERSIDAD METROPOLITANA CASTRO CARAZO

FACULTAD DE INFORMÁTICA

Técnico en Aplicaciones Python / Análisis de Datos

INDAGACIÓN 1 - SEMANA 2

¿Por qué un programa necesita datos correctos y salidas claras?

Nombre del estudiante	Katherine Barrantes Campos
Número de cédula	1-1691-0407
Curso	G02
Docente	Ing. Andrés Mena Abarca
Fecha de entrega	Semana 3, antes de la clase de las 6:00 p. m.

IMPORTANTE - ESTE DOCUMENTO ES LA PLANTILLA DE ENTREGA

Complete las respuestas directamente en los espacios y tablas de este archivo. No cree un informe adicional ni cambie la estructura. Puede ampliar una celda si necesita más espacio, pero mantenga los títulos y el orden establecidos.

Pregunta orientadora

¿Cómo influyen las conversiones numéricas y la presentación de resultados en la confiabilidad de un programa en Python?

Producto de entrega

- Este mismo documento de Word, debidamente completado.
- El archivo del programa Python (.py), elaborado y probado en Thonny.
- Las tres pruebas del programa registradas en la tabla incluida en esta plantilla.

Alcance técnico de la Semana 2

El programa debe ser secuencial. No utilice if, ciclos, listas, funciones ni bibliotecas. Debe solicitar al menos dos datos, convertir los necesarios, realizar una operación aritmética y mostrar un resultado identificable.

1. Justificación

Explique en uno o dos párrafos por qué es importante que un programa reciba datos adecuados y presente resultados comprensibles. Relacione su respuesta con la confiabilidad del programa.

Respuesta del estudiante

Es importante que todos los datos estén claros y tengan una estructura comprensible, porque muchas veces en el futuro será necesario realizar cambios y debemos poder entender lo que hicimos anteriormente. Además, cuando se trata de un archivo compartido, es importante que podamos identificar fácilmente los cambios o aportes realizados por nuestros compañeros de trabajo.

También es importante que la información sea comprensible para otras personas, como los jefes o encargados. Ellos necesitan poder entender rápidamente lo que se está mostrando, especialmente cuando están en reuniones y necesitan consultar algún dato de inmediato. Por eso, mantener la información clara, ordenada y fácil de interpretar facilita el trabajo y evita confusiones.

2. Marco teórico

Responda con sus propias palabras. Sustente las ideas con las fuentes consultadas; no copie definiciones textualmente.

2.1 ¿Qué hace input() en Python?

Explique qué tipo de dato recibe normalmente y para qué se utiliza.

Es la función o expresión encargada de capturar el valor o dato de una variable en especifico que escriba el programador y así poder utilizarla en el código para presentar más delante de acuerdo a las especificaciones del código que se escriba.

2.2 ¿Para qué se utilizan int() y float()?

Explique la diferencia entre ambas conversiones e indique cuándo usaría cada una.

Int es para valores enteros ejemplo números sin decimales, mi entras que ***float*** es para datos que si contengan decimales

2.3 ¿Cuál es la diferencia entre un dato de texto y un dato numérico al realizar cálculos?

Puede incluir un ejemplo sencillo para apoyar la explicación.

El dato de texto no es compatible para operaciones ya que no podemos restar palabras mientras que el dato numérico está diseñado para operaciones.

2.4 ¿Por qué algunos datos recibidos con input() deben convertirse antes de realizar una operación aritmética?

Explique qué podría ocurrir si se intenta calcular utilizando datos que continúan siendo texto.

Debemos indicar previamente como ocupamos el valor de Input, porque el input por si solo reconoce el dato como un texto ejemplo:

base = ***float*** (input('Ingrese la base: '))
altura = ***float*** (input('Ingresa la altura: '))

Si no ponemos ***Float*** o Int nos dará un texto como resultado no un valor numérico

2.5 ¿Por qué una salida debe indicar claramente qué representa el resultado?

Relacione la claridad del mensaje con la posibilidad de comprobar e interpretar el resultado.

Cuando se tiene un código con pocas líneas es posible deducir el resultado fácilmente pero cuando tenemos un código más extenso esto podría complicarse, por eso es importante mantener un código claro y compresible para todos.

3. Programa aplicado

Diseñe un programa secuencial diferente de los ejercicios resueltos en clase. Ejemplos: costo de fotocopias, promedio de notas, área rectangular o gasto de combustible.

Nombre o tema del programa	Formula de la Resistencia
Problema que resolverá	Divide el valor del voltaje entre el valor de la corriente para definir el valor de la resistencia
Datos que solicitará	• Valor que tiene el voltaje • Valor que tiene la corriente

4. Diseño EPS (Entrada - Proceso - Salida)

Complete el diseño antes de programar. Debe coincidir con el archivo .py que entregue.

Entrada	Proceso	Salida
voltaje = float(input("Ingrese el calor del Voltaje: ")) corriente = float(input("Ingrese el calor de la corriente: "))	resistencia = voltaje / corriente	print("La resistencia seria de: ",resistencia)

5. Conclusiones

Responda la pregunta orientadora a partir de lo investigado y de los resultados obtenidos al programar y probar su solución.

Conclusión

(¿Cómo influyen las conversiones numéricas y la presentación de resultados en la confiabilidad de un programa en Python?)

El programa realiza exitosamente la operación para calcular la resistencia (Voltios / Corriente) tal y como se haría normalmente con papel y lápiz, solo que es mas sencillo y mas practico realizarlo con Python

6. Reflexión final

Redacte de 5 a 8 líneas sobre lo aprendido al convertir datos, realizar cálculos, probar el programa y presentar el resultado de manera comprensible.

Reflexión personal

He aprendido que entre más clara sea la escritura de mi código, mejores serán los resultados.
También comprendí que es importante utilizar una buena estructura y organizar correctamente las instrucciones.
La forma en que escribimos el código puede influir en qué tan fácil es entenderlo.
Por eso, debo prestar atención a la gramática y a los nombres que utilizo en las variables.
Además, es importante dedicar tiempo a revisar y corregir posibles errores.
Un código ordenado facilita encontrar problemas y realizar cambios en el futuro, así como verificar todo con la depuración.
También permite que otras personas puedan comprender mejor lo que he realizado.
En conclusión, mientras más claridad y dedicación tenga al programar, mejor expresará mi código lo que quiero que haga.

7. Bibliografía

Incluya al menos tres fuentes confiables. Escriba cada referencia en formato APA. Puede utilizar documentación oficial de Python, recursos educativos institucionales, libros, artículos o sitios de universidades. Un asistente de IA puede apoyar la comprensión, pero no sustituye una fuente académica.

N.º	Referencia en formato APA
1	Bender, A. (2025, abril 29). Fórmula voltios. IBL-Lab GmbH. https://ib-lenhardt.com/kb/glossary/voltage
2	El Libro de Python. (2026). Tipos de datos en Python. https://ellibrodepython.com/tipos-python
3	Sintes Marco, B. (s. f.). Introducción a la programación con Python: Entrada por teclado. mclibre.org. https://www.mclibre.org/consultar/python/lecciones/python-entrada-teclado.html

8. Rúbrica de evaluación

La actividad se califica sobre 8 puntos y la nota obtenida se pondera al 5 % del curso.

Criterio	Evidencia esperada
Conceptos	Explica correctamente input(), conversiones numéricas, tipos de datos y salidas claras.
Fuentes	Utiliza tres o más fuentes confiables y presenta sus referencias en APA.
Diseño EPS	El EPS está completo y coincide con el programa desarrollado.
Programa	El archivo .py funciona en Thonny, es secuencial y respeta el alcance de Semana 2.
Pruebas y reflexión	Presenta tres pruebas completas y una reflexión pertinente de 5 a 8 líneas.

> **Entrega**
>
> Semana 3, antes de iniciar la clase de las 6:00 p. m. El documento debe entregarse completado junto con el archivo .py.